

MongoDB Community Edition – FoodieApp

Assignment Answer

1. Install MongoDB Community Edition and create a database named *foodieApp*

First, MongoDB Community Edition was installed on the system. After installation, the MongoDB shell was opened using the following command:

```
mongosh
```

The **foodieApp** database was then created and selected using:

```
use foodieApp
```

Output:

```
switched to db foodieApp
```

To verify the currently selected database:

```
db
```

Output:

```
foodieApp
```

Thus, the database named **foodieApp** was successfully created/selected.

2. Create a collection named *restaurants* and insert a restaurant document

Inside the **foodieApp** database, a collection named **restaurants** was created by inserting the first restaurant document:

```
db.restaurants.insertOne({
  name: "Spice Villa",
  cuisine: "Indian",
  rating: 4.5,
  location: {
    city: "Surat",
    area: "Vesu"
  }
})
```

Output:

```
Acknowledged: true
InsertedId: ObjectId('...')
```

The above document contains the following fields:

- **name:** Spice Villa
- **cuisine:** Indian
- **rating:** 4.5
- **location:** Surat, Vesu

The **restaurants** collection was automatically created when the first document was inserted.

To verify the collection:

```
show collections
```

Output:

```
restaurants
```

3. Insert three more restaurant documents with an array field

Three additional restaurant documents were inserted into the **restaurants** collection. A **menu** field containing an array of popular dishes was included.

```
db.restaurants.insertMany([
  {
    name: "Pizza Palace",
    cuisine: "Italian",
    rating: 4.2,
    location: {
      city: "Surat",
      area: "Adajan"
    },
    menu: ["Margherita Pizza", "Pasta Alfredo", "Garlic Bread"]
  },
  {
    name: "Tandoori Treat",
    cuisine: "North Indian",
    rating: 4.7,
    location: {
      city: "Surat",
      area: "City Light"
    },
    menu: ["Tandoori Chicken", "Paneer Tikka", "Butter Naan"]
  },
  {
    name: "Wok Express",
    cuisine: "Chinese",
    rating: 4.3,
    location: {
      city: "Surat",
      area: "Piplod"
    },
    menu: ["Hakka Noodles", "Manchurian", "Spring Rolls"]
  }
])
```

Output:

```
Acknowledged: true
Inserted three restaurant documents successfully.
```

Therefore, a total of **four restaurant documents** are now stored in the **restaurants** collection.

4. Retrieve all documents and identify the BSON data types

To retrieve all documents from the **restaurants** collection, the following command was used:

```
db.restaurants.find().pretty()
```

Output:

```
{
  _id: ObjectId('...'),
  name: 'Spice Villa',
  cuisine: 'Indian',
  rating: 4.5,
```

```

    location: {
      city: 'Surat',
      area: 'Vesu'
    }
  }

  {
    _id: ObjectId('...'),
    name: 'Pizza Palace',
    cuisine: 'Italian',
    rating: 4.2,
    location: {
      city: 'Surat',
      area: 'Adajan'
    },
    menu: [
      'Margherita Pizza',
      'Pasta Alfredo',
      'Garlic Bread'
    ]
  }

  {
    _id: ObjectId('...'),
    name: 'Tandoori Treat',
    cuisine: 'North Indian',
    rating: 4.7,
    location: {
      city: 'Surat',
      area: 'City Light'
    },
    menu: [
      'Tandoori Chicken',
      'Paneer Tikka',
      'Butter Naan'
    ]
  }

  {
    _id: ObjectId('...'),
    name: 'Wok Express',
    cuisine: 'Chinese',
    rating: 4.3,
    location: {
      city: 'Surat',
      area: 'Piplod'
    },
    menu: [
      'Hakka Noodles',
      'Manchurian',
      'Spring Rolls'
    ]
  }

```

BSON Types Used

Field	BSON Data Type	Example
_id	ObjectId	ObjectId('...')
name	String	"Spice Villa"
cuisine	String	"Indian"
rating	Number (Double)	4.5
location	Object	{ city: "Surat", area: "Vesu" }

Field	BSON Data Type	Example
menu	Array	["Margherita Pizza", "Pasta Alfredo"]

Explanation of BSON Types

String: The name and cuisine fields contain text values, so their BSON type is String.

Number: The rating field contains numerical values such as 4.5, 4.2, and 4.7, so its BSON type is Number/Double.

Object: The location field contains multiple key-value pairs such as city and area, so it is stored as a BSON Object.

Array: The menu field contains multiple values such as Margherita Pizza and Pasta Alfredo, so it is stored as a BSON Array.

ObjectId: MongoDB automatically creates the `_id` field for every document. Its BSON type is ObjectId and it uniquely identifies each document.

Conclusion

The **foodieApp** database was successfully created with a **restaurants** collection containing four restaurant documents. The documents demonstrate different BSON data types, including **String, Number, Object, Array, and ObjectId**. This shows how MongoDB can store both simple and complex data in a flexible document-oriented structure.